

A Greedy Path-Based Algorithm for Traffic Assignment

Jun Xie¹, Yu (Marco) Nie², and Xiaobo Liu¹

Transportation Research Record
2018, Vol. 2672(48) 36–44
© National Academy of Sciences:
Transportation Research Board 2018
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/0361198118774236
journals.sagepub.com/home/trr
SAGE

Abstract

This paper presents a new path-based algorithm for the static user equilibrium traffic assignment problem. Path-based algorithms are generally considered less efficient than bush-based counterparts, such as Algorithm B, traffic assignment by paired alternative segments (TAPAS), and iTAPAS, an improved version of TAPAS, because explicitly storing and manipulating paths appears wasteful. However, our numerical experiments indicate that the proposed path-based algorithm can outperform TAPAS or iTAPAS by a wide margin. The proposed algorithm, sharing the same Gauss-Seidel decomposition scheme with existing path-based algorithms, delivered a surprising performance, most likely due to its two main features. First, it adopts a greedy method to solve the restricted subproblem defined on each origin–destination (O-D) pair. Second, instead of sequentially visiting every O-D pair in each iteration, it introduces an intelligent scheme to determine which O-D pairs need more or less work. The proposed algorithm is also more straightforward to implement than bush-based algorithms.

The user equilibrium (UE) static traffic assignment problem (TAP) has long been used as a standard tool to predict network flows (1). Under mild assumptions, the UE-TAP can be formulated as a convex optimization program (2–4). Designing efficient solution algorithms for this problem in large regional-scale networks has been a recurring research theme in transportation science and has attracted much attention in the past decades.

The Frank and Wolfe (FW) algorithm (5) was the first widely applied convergent algorithm for UE-TAP. Easy to implement and fast to converge in early iterations, the FW algorithm has emerged as the primary TAP-solver since the 1970s. However, the FW algorithm is not designed to achieve high-quality solutions that are often deemed necessary in modern planning practice (6). A number of strategies have been proposed to accelerate the FW algorithm. Well-known refinements include the parallel tangent algorithm (7), the aggregated simplicial decomposition algorithm (8) and the conjugate Frank–Wolfe algorithm (9). Since FW and its improved variants all operate in the space of link flows, they are usually called link-based algorithms.

The ability to solve UE-TAP with high precision has been enhanced substantially with the development of bush-based algorithms in the past decade. Simply speaking, a bush is an acyclic subnetwork that intends to encompass all the UE shortest paths from a given origin. Typically, a bush-based algorithm constructs and

maintains a bush for each origin (or destination) and restricts the assignment operation only to these bushes. The first algorithm of this class is the origin-based algorithm (OBA) (10), which sequentially solves a list of node-based subproblems defined on the bush rooted at each origin by a gradient projection (GP) method. As a variant of OBA, the local user cost equilibrium (LUCE) (11) solves the node-based subproblems with a greedy method instead of GP. Algorithm B (12) simultaneously finds the min- and max-path trees for a given origin; it then equalizes the bush rooted at the origin by shifting flows from the maximum-cost paths to the minimum-cost ones. In traffic assignment by paired alternative segments (TAPAS) (13), a pair of alternative segments (PAS) is defined as two completely disjoint path segments with the same starting and ending nodes. Thus, a PAS marks the distinct parts of two or more paths connecting one origin–destination (O-D) pair. The notable innovation of TAPAS is to equalize path costs through identifying, storing, and operating a list of PASs, which

¹School of Transportation and Logistics, Southwest Jiaotong University, Chengdu, China

²Department of Civil and Environmental Engineering, Northwestern University, Evanston, IL

Corresponding Author:

Address correspondence to Jun Xie: jun.xie@swjtu.edu.cn

avoids unnecessary network scanning. An improved version of TAPAS, named iTAPAS (14), employs a new PAS identification method and associates each PAS with only one origin to speed up the convergence efficiency. For a more comprehensive comparison of bush-based algorithms the reader is referred to Nie, and Xie and Xie (15, 16).

UE-TAP can also be solved by path-based algorithms.

This class of algorithms decomposes the original TAP into a list of subproblems by O-D pairs, which are then solved sequentially in the space of path flows. Whenever one subproblem is solved, the path flows from all other O-D pairs are usually fixed. Typically, a path-based algorithm needs to store and maintain a set of paths for each O-D pair, which can be iteratively generated through a process known as column generation. Several path-based algorithms have been proposed in the literature, which differ from each other mainly in the solution method for the restricted subproblem. Dafermos and Sparrow proposed to solve the subproblems by shifting flows from the longest path to the current shortest path in the path set (17); Larsson and Patriksson defined and solved an auxiliary optimization problem for each O-D pair (18); Jayakrishnan et al. and Florian et al. adapted the GP and projected gradient (PG) methods respectively to solve the subproblems (19, 20). Two limitations had probably hindered the application of the path-based algorithms in solving large-scale problems in the past: on the one hand, storing and manipulating paths for each O-D pair needs much more random access memory (RAM) than link-based algorithms, which is a crucial issue when RAM is scarce; on the other hand, the decomposition scheme by O-D pairs generates a large number of subproblems, and sequentially solving them with equivalent effort usually leads to a relatively low overall convergence efficiency.

The research motivation of this study was three-fold. First, with the rapid advance in computing power and storage capacity, RAM usage of path-based algorithms is no longer a critical issue for modern-size regional planning networks. Thus, the path-storing requirement should not limit the development of faster traffic assignment algorithms. Second, despite their promising efficiency, most bush-based algorithms are complex in terms of both methodology and implementation, which makes them less accessible to researchers and practitioners. **In contrast, path-based algorithms rely on a decomposition scheme that is easy to understand and implement, and they are equally capable of achieving high-precision solutions.** Third, much has been learned from developing bush-based algorithms in the past decade, and these lessons can now be adapted to reinventing path-based algorithms. Indeed, recent research (21) observed that the efficiency of the path-based algorithm can be improved

by only skipping column generation in certain iterations. In a nutshell, the time is ripe to give the path-based algorithm a fresh look.

The new path-based algorithm developed in this paper shares the same Gauss-Seidel decomposition scheme with existing path-based algorithms, in which O-D pairs are visited sequentially. Yet, two main innovative features differentiate it from the path-based algorithms in the literature. First, the greedy method, which serves as the solver of node-based subproblems in LUCE, **is adopted to solving the restricted subproblem defined on each O-D pair.** The greedy method is generally faster than GP in solving the O-D subproblems (22). **Second, instead of sequentially visiting each O-D pair in each iteration, it introduces an intelligent scheme to determine which O-D pairs need more or less work.** Specifically, an inner loop is added to the main loop to make sure that the frequency of path flow adjustment is higher than that of column generation. Moreover, **the algorithm tracks the maximum path cost difference for each O-D pair and repeats the path flow adjustment more frequently on those O-D pairs with a larger cost difference.** We also improve the computational efficiency by properly setting a maximum convergence precision in each iteration, **that is, skipping the path flow adjustment for the O-D pairs with a cost difference smaller than the current relative convergence gap.** Our numerical experiments show that the proposed path-based algorithm is significantly faster than the state-of-the-art bush-based algorithms.

The remainder of this paper is organized as follows. Section 2 presents the notation and formulation required. Section 3 describes the algorithm details and implementation procedures. Section 4 demonstrates the high efficiency of the proposed algorithm by comparing it with several of the latest bush-based algorithms. The last section concludes the paper and provides directions for future research.

Notation and Formulation

Consider a directed transportation network $G(N, A)$ where N and A denote the sets of nodes and links, respectively. Suppose that G is strongly connected, that is, there is at least one path between any two nodes in G . N consists of a set of origins R and a set of destinations S . The travel demand between an origin $r \in R$ and a destination $s \in S$ is denoted by d_{rs} . Let H_{rs} denote the set of simple paths connecting O-D pair (r, s) and f_h the flow on path $h \in H_{rs}$. A link is defined by (i, j) with i and j being the tail and head node respectively. The general cost of traversing link (i, j) , denoted as $t_{ij}(x_{ij})$, is assumed to be a separable, strictly positive, and monotonically increasing function of x_{ij} , the flow on link (i, j) .

Under the above assumptions the TAP satisfying the UE conditions (or Wardrop's first principle) can be formulated mathematically as the following **convex program**:

$$\min z(f) = \sum_{(i,j) \in A} \int_0^{x_{ij}} t_{ij}(w) dw \quad (1)$$

subject to

$$\sum_{h \in H_{rs}} f_h = d_{rs}, \quad \forall r \in R, \forall s \in S, \quad (2)$$

$$f_h \geq 0, \quad \forall h \in H_{rs}, \forall r \in R, \forall s \in S, \quad (3)$$

$$x_{ij} = \sum_{r \in R} \sum_{s \in S} \sum_{h \in H_{rs}} \delta_{ij}^h f_h, \quad \forall (i,j) \in A, \quad (4)$$

where $\delta_{ij}^h = 1$ if path h passes link (i,j) ; and 0 otherwise. It is well known that the UE-TAP is strictly convex given the above mild assumption on $t_{ij}(x_{ij})$ and hence admits a unique solution in the space of link flows. However, its path flow solution is not unique.

Central to the development of path-based algorithms is a sequential (known as Gauss-Seidel) decomposition of the above problem with respect to O-D pairs. As the decomposed subproblems are solved sequentially, it suffices for our purpose to focus on a single O-D subproblem, which may be formulated as follows:

$$\min z_{rs}(f) = \sum_{(i,j) \in A} \int_0^{x_{ij}^- + x_{ij}^{rs}} t_{ij}(w) dw \quad (5)$$

subject to

$$\sum_{h \in H_{rs}} f_h = d_{rs}, \quad (6)$$

$$f_h \geq 0, \quad \forall h \in H_{rs}, \quad (7)$$

$$x_{ij}^{rs} = \sum_{h \in H_{rs}} \delta_{ij}^h f_h, \quad \forall (i,j) \in A, \quad (8)$$

where $x_{ij}^- = \sum_{o \in R, o \neq r} \sum_{q \in S, q \neq s} x_{ij}^{oq}$ are flows contributed by all O-D pairs other than (r,s) . To solve the above non-linear program, one can construct a quadratic approximation for it to provide a descent direction. At the current solution $\{g_h, \forall h \in H_{rs}, \forall r \in R, \forall s \in S\}$, the objective function (5) can be approximated by the second-order Taylor expansion as follows:

$$\begin{aligned} \hat{z}_{rs}(f) \simeq & z_{rs}(g) + \sum_{h \in H_{rs}} \left(\frac{\partial z_{rs}(g)}{\partial f_h} (f_h - g_h) \right. \\ & \left. + 0.5 \frac{\partial^2 z_{rs}(g)}{\partial^2 f_h} (f_h - g_h)^2 \right). \end{aligned} \quad (9)$$

Let $v_h^g = \frac{\partial z_{rs}(g)}{\partial f_h}$, $s_h^g = \frac{\partial^2 z_{rs}(g)}{\partial^2 f_h}$ and remove the constants from formulation (9), the quadratic approximation of the single O-D assignment problem can then be formulated as follows:

$$\min \hat{z}_{rs}(f) = \sum_{h \in H_{rs}} ((v_h^g - s_h^g g_h) f_h + 0.5 s_h^g f_h^2) \quad (10)$$

subject to

$$\sum_{h \in H_{rs}} f_h = d_{rs}, \quad (11)$$

$$f_h \geq 0, \quad \forall h \in H_{rs}. \quad (12)$$

Let w_{rs} be the multiplier associated with O-D pair (r,s) , then the Karush-Kuhn-Tucker conditions of problem (10–12) indicate,

$$v_h^g + s_h^g (f_h - g_h) - w_{rs} \geq 0, \quad (13)$$

$$f_h (v_h^g + s_h^g (f_h - g_h) - w_{rs}) = 0. \quad (14)$$

Here w_{rs} should be interpreted as the minimum travel cost between r and s , and the quadratic approximation problem in fact approximates v_h with the following linear function at the current solution g ,

$$\hat{v}_h = v_h^g + s_h^g (f_h - g_h). \quad (15)$$

where v_h^g and s_h^g can be easily calculated precisely as follows:

$$v_h^g = \frac{\partial z_{rs}(g)}{\partial f_h} = \sum_{(i,j) \in A} t_{ij}(x_{ij}^g) \delta_{ij}^h, \quad (16)$$

$$s_h^g = \frac{\partial^2 z_{rs}(g)}{\partial^2 f_h} = \sum_{(i,j) \in A} \frac{\partial t_{ij}(x_{ij}^g)}{\partial x_{ij}^g} \delta_{ij}^h. \quad (17)$$

Many methods can be used to solve the quadratic approximation program (10–12). In the literature, the Goldstein–Levitin–Polyak GP method and Rosen's PG – known respectively as the GP algorithm and the PG algorithm in traffic assignment – were adapted respectively by Jayakrishnan et al. and Florian et al. (19, 20). In the next section, we will introduce the greedy method to solve the above quadratic program, followed by an implementation.

Algorithm Details

Adaption of the Greedy Method

The greedy method was first used in the hyper-path algorithm for transit assignment (23, 24), and then adapted by Gentile in the LUCE algorithm (11) to solve the node-

based assignment problems. In what follows we adopt this method to solve the single O-D assignment problem.

Let us first simplify our notation by introducing the following constant:

$$c_h^g \equiv v_h^g - s_h^g g_h. \quad (18)$$

Suppose we know the set of all used paths $\hat{H}_{rs}$ between r and s ; then the conditions (13)–(14) can be translated to the following equation system.

$$c_h^g + s_h^g f_h = \bar{w}_{rs}, \quad \forall h \in \hat{H}_{rs}, \quad (19)$$

$$\sum_{h \in \hat{H}_{rs}} f_h = d_{rs}. \quad (20)$$

From the above system, one can solve the minimum average path travel time $\bar{w}_{rs}$ and f_h as follows:

$$\bar{w}_{rs} = \frac{d_{rs} + \sum_{h \in \hat{H}_{rs}} c_h^g / s_h^g}{\sum_{h \in \hat{H}_{rs}} 1 / s_h^g}, \quad (21)$$

$$f_h = \bar{w}_{rs} / s_h^g - c_h^g / s_h^g, \quad \forall h \in \hat{H}_{rs}. \quad (22)$$

All paths in $H_{rs} \setminus \hat{H}_{rs}$ should receive zero flow. To determine $\hat{H}_{rs}$, the procedure described in Algorithm 1 is used.

Algorithm 1 consists of three parts: initialization (cf. lines 3–9), main loop (cf. lines 11–17), and flow update (cf. lines 19–31). In the initialization part, all paths in the set H_{rs} should be sorted according to the increasing order of c_h^g , and then $\bar{w}_{rs}$ is initialized using the path with the least c_h^g (cf. Line 8). In the worst case, the main loop part will find the exact solution to the quadratic program (10–12) after all paths in H_{rs} are visited, which is faster than the GP method when there are more than two used paths in H_{rs} . Note that the typical GP [e.g. the one presented in Jayakrishnan et al. (19)] can only give an approximate solution to the quadratic program in this situation. The reader is referred to Xie et al. (22) for a comparison between the two methods in solving a quadratic program. Finally, the flow update part updates the flow of the paths in H_{rs} , as well as the flow and cost of the links on those paths. Note that, in our current implementation, only the links on the paths subject to flow change are updated (cf. Line 26 of Algorithm 1). Whether these paths are overlapping or not is not considered. Furthermore, at the end of Algorithm 1 the paths receiving zero flow are all removed from the path set H_{rs} (cf. Line 31 of Algorithm 1).

Algorithm Implementation

The implementation of the greedy path-based algorithm is described in Algorithm 2. In addition to the greedy-method solver for O-D subproblems, another noteworthy improvement was an efficient implementation

Algorithm 1 Adaption of the greedy method for a single O-D assignment problem

```

1: Input: The current solution  $\{g_h, \forall h \in H_{rs}\}$  is given.
2: Initialization: lines 3–9.
3: for each  $h \in H_{rs}$  do
4:   Compute  $v_h^g$  and  $s_h^g$  according to formulation (16) and
     formulation (17), respectively.
5:   Compute  $c_h^g$  according to formulation (18).
6: end for
7: Sort all paths  $h \in H_{rs}$  according to the increasing order of  $c_h^g$ ,
   i.e.,  $H_{rs} = \{1, 2, 3, \dots\}$  with  $c_1^g \leq c_2^g \leq c_3^g \leq \dots$ 
8: Let  $B = 1/(s_1^g d_{rs})$ ,  $C = c_1^g/(s_1^g d_{rs})$  and  $\bar{w}_{rs} = (1.0 + C)/B$ .
9: Set  $h = 2$ ,  $\hat{H}_{rs} = \{1\}$ .
10: Main Loop: lines 11–17.
11: while  $h \leq |H_{rs}|$  and  $c_h^g < \bar{w}_{rs}$  do
12:   Set  $C = C + c_h^g/(s_h^g d_{rs})$ .
13:   Set  $B = B + 1/(s_h^g d_{rs})$ .
14:   Set  $\bar{w}_{rs} = (1.0 + C)/B$ .
15:   Let  $\hat{H}_{rs} = \hat{H}_{rs} \cup \{h\}$ .
16:   Set  $h = h + 1$ .
17: end while
18: Flow Update: lines 19–31.
19: end for
20: for each  $h \in \hat{H}_{rs}$  do
21:   Set  $f_h = (\bar{w}_{rs} - c_h^g)/s_h^g$ .
22: for each  $h \in H_{rs} \setminus \hat{H}_{rs}$  do
23:   Set  $f_h = 0$ .
24: end for
25: for each  $h \in H_{rs}$  do
26:   if  $f_h \neq g_h$  then
27:     Update the link flow by  $x_{ij} = x_{ij} + (f_h - g_h), \forall (i, j) \in h$ .
28:     Update  $t_{ij}(x_{ij})$  and  $\frac{\partial t_{ij}(x_{ij})}{\partial x_{ij}}$  for each link  $(i, j) \in h$ .
29:   end if
30: end for
31: Let  $H_{rs} = \hat{H}_{rs}$ .
32: Output: A new solution  $\{f_h, \forall h \in H_{rs}\}$ .

```

scheme, which was designed to expedite the overall convergence of path-based algorithms mainly from three aspects. First, an inner loop (cf. lines 18–34 of Algorithm 2) was added to make sure that the frequency of path flow adjustment (by Algorithm 1) was higher than that of column generation. Second, it tracked the maximum path cost difference for each O-D pair, that is, $\Delta_{rs} = \max\{v_h\} - \min\{v_h\}, \forall h \in H_{rs}$, and repeated the path flow adjustment more frequently on the O-D pairs with larger Δ_{rs} (cf. lines 22–29 of Algorithm 2). Third, it improved computational efficiency by properly setting a maximum convergence precision in each iteration, that is, skipping the path flow adjustment for those O-D pairs with $\Delta_{rs} < RG^{k-1}/2.0$, where RG^{k-1} is the relative convergence gap in the last iteration (cf. lines 25–29 of Algorithm 2). The above three improvements in implementation ensured that more computational resources were allocated to handle those O-D pairs that can lead to more reduction of the objective value.

Algorithm 2 The greedy path-based algorithm

```

1: Initialize: lines 2–7
2: for each O-D pair  $(r, s)$  do
3:   Compute the shortest path  $\hat{h}$  from  $r$  to  $s$ .
4:   Assign all  $d_{rs}$  to  $\hat{h}$  and push it into  $H_{rs}$ .
5: end for
6: Update link flows by
    $x_{ij} = \sum_{r \in R} \sum_{s \in S} \sum_{h \in H_{rs}} f_h \delta_{ij}^h, \quad \forall (i, j) \in A$ .
7: Update the link cost  $t_{ij}(x_{ij})$  and its derivative  $\frac{\partial t_{ij}(x_{ij})}{\partial x_{ij}}$  for each link
    $(i, j) \in A$ .
8: Main Loop: lines 9–34
9: for each  $r \in R$  do
10:   Compute the shortest path tree from  $r$  to all its
       destinations  $S_r$ .
11:   for each  $s \in S_r$  do
12:     Build the shortest path  $\hat{h}$  from  $r$  to  $s$ .
13:     Push  $\hat{h}$  into  $H_{rs}$  if  $\hat{h} \notin H_{rs}$ ; otherwise delete  $\hat{h}$ .
14:     Perform path flow adjustment for  $H_{rs}$  by Algorithm 1.
15:   end for
16: end for
17: Inner Loop: lines 18–34
18: Set  $l = 0, \text{Max}l = 1000, FC = 0$ .
19: while  $l < \text{Max}l$  do
20:   Let  $l = l + 1$  and  $FC = 0$ .
21:   for each O-D pair  $(r, s)$  do
22:     if  $l \% 100 = 0$  then
23:       Compute  $\Delta_{rs} = \max\{v_h\} - \min\{v_h\}, \forall h \in H_{rs}$ .
24:     end if
25:     if  $\Delta > RG^{k-1}/2.0$  then
26:       Let  $FC = FC + 1$ .
27:       Perform path flow adjustment for  $H_{rs}$  by Algorithm 1.
28:     Update  $x_{ij}, t_{ij}(x_{ij})$  and  $\frac{\partial t_{ij}(x_{ij})}{\partial x_{ij}}$  for  $\forall (i, j) \in h, \forall h \in H_{rs}$ .
29:   end if
30: end for
31: if  $FC = 0$  then
32:   Break the inner loop.
33: end if
34: end while
35: Convergence Check: If the convergence criterion is
       achieved, stop; otherwise, repeat the Main Loop.

```

In the current implementation, several parameters were set according to the limited experience gained from the computational experiments on the networks shown in the next section. First, in Line 26 of Algorithm 2, we set a maximum convergence precision for each O-D subproblem as $RG^{k-1}/2.0$, which was sufficient to ensure a consecutive decrease of the objective value in each main loop. Second, in Line 19 of Algorithm 2, the maximum allowed inner loop iteration was set at 1,000, trying to enforce all O-D pairs to reach the desired convergence precision. Third, in lines 22–24, the Δ_{rs} for each O-D pair was updated only when the inner loop index l was out by a factor of 100; otherwise, Δ_{rs} was kept as the value obtained in the last update. Fourth, we created a

parameter FC to track how many O-D pairs were updated by Algorithm 1 in each inner loop (cf. lines 18–20 and 26 in Algorithm 2); if $FC = 0$ (cf. lines 31–33 in Algorithm 2), which generally means that all O-D pairs have achieved the desired convergence precision in the current main loop, then we should break the inner loop.

Finally, a very simple data structure was employed in the current implementation to keep and operate paths. Basically, we maintained a vector H_{rs} for each O-D pair $(r, s), \forall r \in R, s \in S$, which was initialized with the shortest path from r to s in the beginning (cf. Line 4 of Algorithm 2); and then it was expanded when there was a new path generated in the column generation (cf. Line 13 of Algorithm 1), or reduced when there were paths receiving zero flow in the flow assignment (cf. lines 22–24 and Line 31 of Algorithm 1).

Numerical Results

In this section, we compare the convergence of the proposed greedy path-based algorithm with that of TAPAS, iTAPAS, GP, and an improved GP (iGP for short) on computing four testing networks. GP was implemented exactly following the procedure provided in Mitradjeva and Lindberg (19); the iGP was an implementation of GP plus the proposed intelligent scheme, that is, a replication of Algorithm 2 by replacing the greedy solver (cf. Line 14) with a GP solver; TAPAS and iTAPAS were implemented following the procedures in Xie and Xie (14). All these algorithms were coded using the network modeling toolkit, a C++ class library specialized in modeling transportation networks. Different algorithms shared as many subroutines as they could to ensure a fair comparison. All numerical results reported in this section were produced on a Windows 10 64-bit workstation with Intel Xeon central processing unit (CPU) E3-1225 V3 3.3 GHz CPU and 16G RAM.

The algorithms were applied to the four testing networks obtained from <http://www.bgu.ac.il/~bargera/ntnp/>. The properties of the four networks are given in Table 1. The convergence indicator used in these experiments was the relative gap (RG), which was calculated as follows:

$$RG = 1 - \frac{\sum_{rs} \mu_{rs} d_{rs}}{\sum_{(i,j) \in A} x_{ij} t_{ij}(x_{ij})} \quad (23)$$

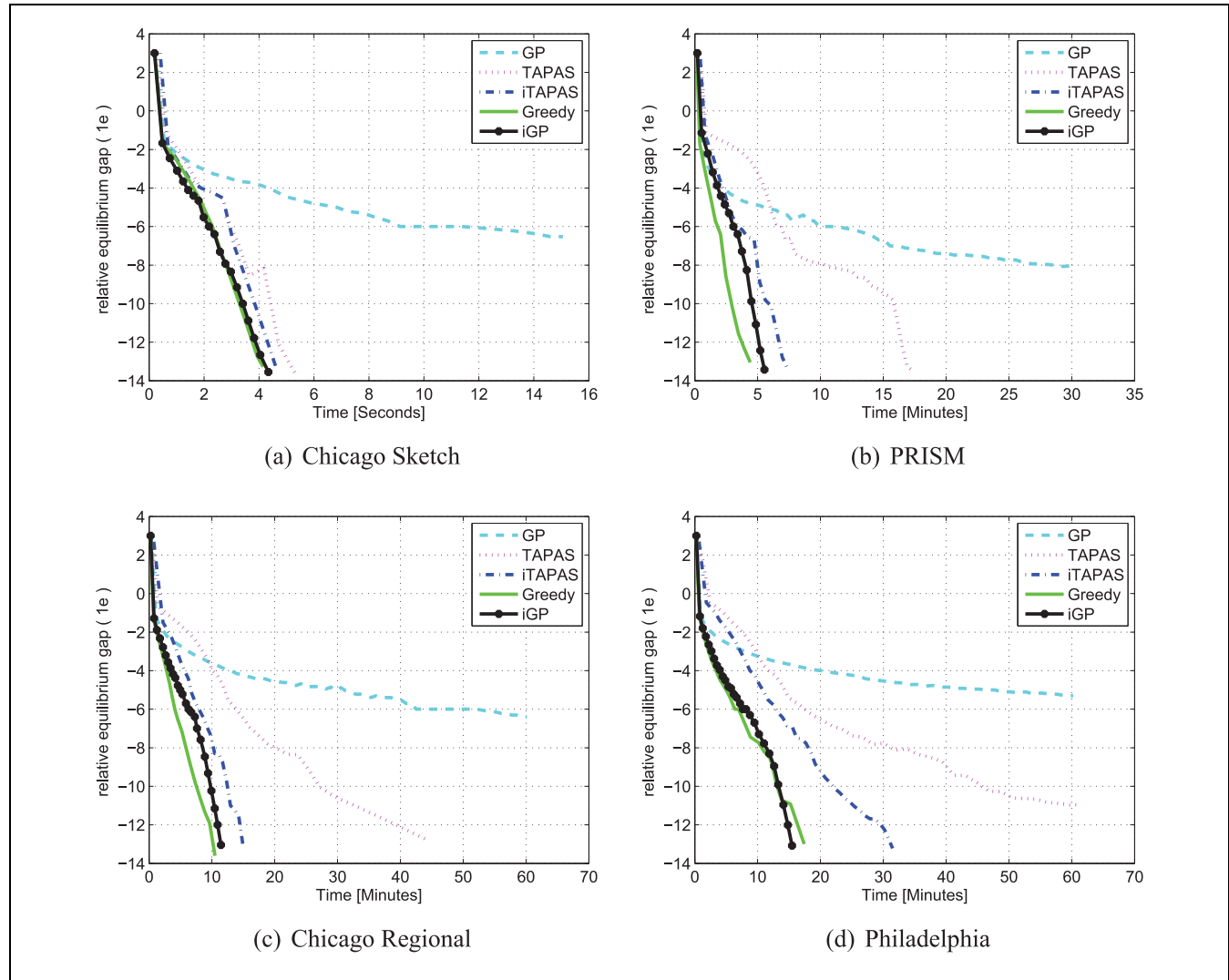
where μ_{rs} is the minimum travel cost between O-D pair (r, s) . Finally, the Bureau of Public Roads function was used to calculate link costs.

The computational results for all the testing networks with the five algorithms are shown in Figure 1. All the testing algorithms were stopped when $RG = 10^{-14}$ was achieved or 1 h had passed.¹ In these plots, the horizontal

Table 1. Test Networks Used in the Numerical Experiments

Networks	Nodes	Links	Zones	O-D Pairs	V/C
Chicago Sketch	933	2950	386	1,260,910	0.54
PRISM	14,639	33,937	898	609,670	0.26
Chicago Regional	12,982	39,018	1771	3,136,441	0.60
Philadelphia	13,389	40,223	1525	1,150,722	0.75

Note: V/C = volume/capacity.

**Figure 1.** Convergence performance comparison on computing selected test networks.

axis represents CPU times, and the vertical axis represents the RG. Generally, the proposed greedy algorithm could outperform iTAPAS, TAPAS, and GP with a wide margin in all testing networks. Specifically, for the Chicago Regional network (cf. Figure 1c), TAPAS and iTAPAS took about 38 min and 14 min respectively to achieve $RG = 10^{-12}$, and the greedy algorithm took only 9 min to

converge to the same precision, which is about 55% faster than iTAPAS and four times faster than TAPAS. In contrast, the GP algorithm only converged to about $RG = 10^{-7}$ within 1 h.² Overall, the same comparison result that the greedy algorithm is faster than the GP, TAPAS, and iTAPAS algorithms was obtained from the other three testing networks. These results generally

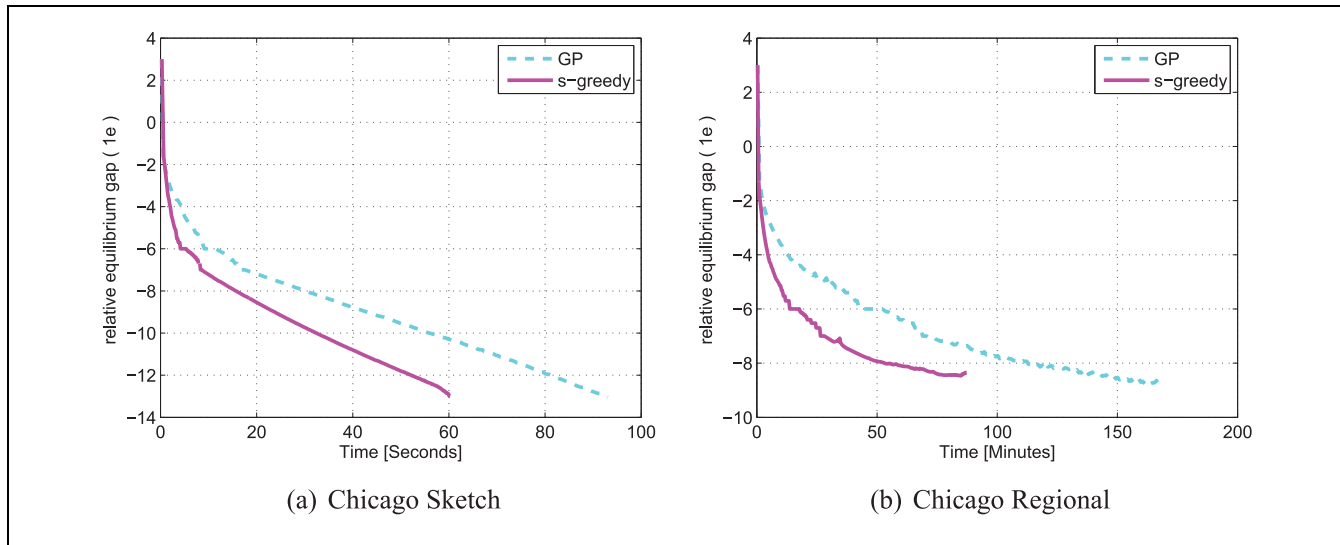


Figure 2. Comparison on the convergence performance of s-greedy and GP.

Table 2. Path Information for Different Algorithms

Networks	Information	GP	greedy	iGP
Chicago Regional	path number	1,985,744	1,991,055	1,920,298
	memory size	489 M	491 M	473 M
Philadelphia	path number	1,712,502	1,709,818	1,376,289
	memory size	583 M	562 M	437 M

Note: M = megabyte.

indicate that the proposed path-based algorithm is faster than the state-of-the-art bush-based algorithms.

The proposed intelligent scheme for path-based algorithms (cf. Algorithm 2) plus the GP solver, iGP, was also tested in these numerical examples. Interestingly, iGP and the greedy algorithm performed very similarly in all experiments. Taking a closer look, the greedy algorithm performed a little better than iGP in computing the PRISM (this model of the Birmingham City Region in England is a modified version of the Prism model, described at <http://www.prism-wm.com/>) and the Chicago Regional, but iGP was still ahead of the other three algorithms in the whole convergence process for all experiments. Recall that the major difference between the GP method and the greedy method for solving a quadratic program lies in the fact that the former can obtain an exact solution, whereas the later can only get an approximate solution when there are more than two used paths in the feasible set. Yet, we noticed that GP was also able to reach the same convergence precision as the greedy method does through repeating itself multiple iterations, that is, performing the flow shifting operations as many times as required to converge to the desired level of precision.³ Given the above background, a reasonable explanation for the similar performance between iGP and the

greedy method may be that, the proposed intelligent scheme, which is designed to allow the GP method or the greedy method to repeat multiple times to reach the desired convergence precision, thoroughly makes up for the inefficient convergence of the GP method.

To further support this explanation, we implemented the greedy method in the same scheme with GP, that is, a procedure of Algorithm 2, removing the inner loop. Such a simplified greedy algorithm is named s-greedy. The convergence performance of s-greedy and GP in solving Chicago Sketch and Chicago Regional is shown in Figure 2. Clearly, s-greedy is more efficient than GP in these two examples. For the Chicago Sketch network, s-greedy was over 60% faster than GP to achieve $RG = 10^{-12}$; and for the Chicago Regional network, s-greedy was about twice as fast as GP to achieve $RG = 10^{-8}$. As a result, it is safe to conclude that both the implementation strategies proposed in this paper contribute positively to enhancing the computational performance of the path-based algorithms.

Table 2 reports the number of paths generated by each path-based algorithm for the two regional testing networks and the corresponding memory (M represents megabyte) required to save them in the computer. Overall, all three path-based algorithms generated less

than 2 million used paths in the two networks, which required less than 600 M memory to save in the computer. Such a requirement on memory is clearly no longer a hurdle for most PCs.

Conclusion

An efficient implementation of a path-based algorithm is proposed for the static UE-TAP. It retains the simplicity of path-based algorithms, while taking advantage of the lessons learned from developing bush-based algorithms in the past decade. The proposed algorithm adapts a greedy method to solve the restricted subproblem for each O-D pair, and introduces an intelligent scheme to determine which O-D pairs need more or less work in each iteration. These features make the new algorithm about 55% faster than iTAPAS, or more than four times faster than TAPAS in solving large regional-scale problems, according to our numerical experiments.

Our experience has revealed several promising generic rules for designing efficient path-based UE-TAP algorithms. First, the frequency of path flow adjustment should be significantly higher than that of column generation. Second, one should try to repeat more flow adjustment on the less converged O-D pairs. Third, keep in mind that each subproblem is only subject to a local optimal solution, hence do not waste time seeking high-precision solutions to subproblems in early iterations.

As the potential of path-based algorithms for the standard static UE-TAP has been well demonstrated here, a future research direction could be to extend this to solving more sophisticated models, such as stochastic, multi-class, and asymmetric TAPs.

Acknowledgments

The work was conducted when the first author visited Northwestern University as a visiting postdoctoral researcher. He was funded by Chinese National Nature Science Foundation (Grant No. 71501129) and Chinese International Postdoctoral Exchange Fellowship Program (No. 20150045). The work was also partially funded by the United States National Science Foundation under the award number CMMI-1402911.

Author Contributions

The authors confirm contribution to the paper as follows: study conception and design: J.X.; data collection: J.X.; analysis and interpretation of results: J.X., Y.M.N., X.L.; draft manuscript preparation: J.X., Y.M.N., X.L. All authors reviewed the results and approved the final version of the manuscript.

Notes

1. For the Chicago Sketch and PRISM networks, different convergence time intervals, that is, 15 s and 30 min respectively, are shown in the figures to highlight the difference.

2. Note that the GP algorithm can converge better with time elapses.
3. Refer to Section 6 of Xie et al. (22) for more details on the comparison of GP and greedy.

References

1. Wardrop, J. G. Some Theoretical Aspects of Road Traffic Research. *Proceedings of the Institution of Civil Engineers*, Vol. 1, No. 3, 1952, pp. 325–362.
2. Beckmann, M., C. McGuire, and C. B. Winsten. *Studies in the Economics of Transportation*. Yale University Press, New Haven, 1956.
3. Sheffi, Y. *Urban Transportation Network*. Prentice Hall, Englewood Cliffs, N.J., 1985.
4. Patriksson, M. *The Traffic Assignment Problem: Models and Methods*. Courier Dover Publications, New York, 2015.
5. Frank, M., and P. Wolfe. An Algorithm for Quadratic Programming. *Naval Research Logistics Quarterly*, Vol. 3, No. 1–2, 1956, pp. 95–110.
6. Boyce, D., B. Ralevic-Dekic, and H. Bar-Gera. Convergence of Traffic Assignments: How Much is Enough? *Journal of Transportation Engineering*, Vol. 130, No. 1, 2004, pp. 49–55.
7. Florian, M., J. Guálat, and H. Spiess. An Efficient Implementation of the Partan Variant of the Linear Approximation Method for the Network Equilibrium Problem. *Networks*, Vol. 17, No. 3, 1987, pp. 319–339.
8. Lawphongpanich, S., and D. W. Hearn. Simplicial Decomposition of the Asymmetric Traffic Assignment Problem. *Transportation Research Part B: Methodological*, Vol. 18, No. 2, 1984, pp. 123–133.
9. Mitradjieva, M., and P. O. Lindberg. The Stiff is Moving-conjugate Direction Frank-Wolfe Methods with Applications to Traffic Assignment. *Transportation Science*, Vol. 47, No. 2, 2013, pp. 280–293.
10. Bar-Gera, H. Origin-Based Algorithm for the Traffic Assignment Problem. *Transportation Science*, Vol. 36, No. 4, 2002, pp. 398–417.
11. Gentile, G. Local User Cost Equilibrium: A Bush-Based Algorithm for Traffic Assignment. *Transportmetrica A: Transport Science*, Vol. 10, No. 1, 2014, pp. 15–54.
12. Dial, R. B. A Path-Based User-Equilibrium Traffic Assignment Algorithm that Obviates Path Storage and Enumeration. *Transportation Research Part B: Methodological*, Vol. 40, No. 10, 2006, pp. 917–936.
13. Bar-Gera, H. Traffic Assignment by Paired Alternative Segments. *Transportation Research Part B: Methodological*, Vol. 44, No. 8, 2010, pp. 1022–1046.
14. Xie, J., and C. Xie. New Insights and Improvements of Using Paired Alternative Segments for Traffic Assignment. *Transportation Research Part B: Methodological*, Vol. 93, 2016, pp. 406–424.
15. Nie, Y. M. A Class of Bush-Based Algorithms for the Traffic Assignment Problem. *Transportation Research Part B: Methodological*, Vol. 44, No. 1, 2010, pp. 73–89.
16. Xie, J., and C. Xie. Origin-Based Algorithms for Traffic Assignment: Algorithmic Structure, Complexity Analysis,

- and Convergence Performance. *Transportation Research Record: Journal of the Transportation Research Board*, 2015. 2498: 46–55.
17. Dafermos, S. C., and F. T. Sparrow. The Traffic Assignment Problem for a General Network. *Journal of Research of the National Bureau of Standards B*, Vol. 73, No. 2, 1969, pp. 91–118.
 18. Larsson, T., and M. Patriksson. Simplicial Decomposition with Disaggregated Representation for the Traffic Assignment Problem. *Transportation Science*, Vol. 26, No. 1, 1992, pp. 4–17.
 19. Jayakrishnan, R., W. T. Tsai, J. N. Prashker, and S. Rajadhyaksha. *A Faster Path-Based Algorithm for Traffic Assignment*. University of California Transportation Center, 1994.
 20. Florian, M., I. Constantin, and D. Florian. A New Look at Projected Gradient Method for Equilibrium Assignment. *Transportation Research Record: Journal of the Transportation Research Board*, 2009. 2090: 10–16.
 21. Galligari, A., and M. Sciandrone. A Convergent and Fast Path Equilibration Algorithm for the Traffic Assignment Problem. *Optimization Methods and Software*, Vol. 33, No. 2, 2017, pp. 354–371.
 22. Xie, J., Y. M. Nie, and X. Yang. Quadratic Approximation and Convergence of Some Bush-Based Algorithms for the Traffic Assignment Problem. *Transportation Research Part B: Methodological*, Vol. 56, 2013, pp. 15–30.
 23. Nguyen, S., and S. Pallottino. Equilibrium Traffic Assignment for Large Scale Transit Networks. *European Journal of Operational Research*, Vol. 37, No. 2, 1988, pp. 176–186.
 24. Spiess, H., and M. Florian. Optimal Strategies: A New Assignment Model for Transit Networks. *Transportation Research Part B: Methodological*, Vol. 23, No. 2, 1989, pp. 83–102.
- The Standing Committee on Transportation Network Modeling (ADB30) peer-reviewed this paper (18-03128).*